Fachhochschule Wedel

Project IT Engineering

Optimizing ArUCO Marker Detection on Nvidia Jetson TX2 Using Cuda

Submitted by: Chou, Li-Chieh

Field of Study: IT Engineering

Matriculation Number: ITE104934

Submitted to: Hermann Höhne

Contents

1. Introduction

Real-time image processing is a critical factor in applications ranging from autonomous vehicles to medical imaging. One of the widely used libraries for this purpose is OpenCV, which provides a comprehensive set of tools for image processing and computer vision tasks. Among these tools, the ArUCO marker detection is particularly useful for applications requiring precise object localization, such as augmented reality, robotics, and camera calibration.

This project focuses on leveraging CUDA, a parallel computing platform and application programming interface (API) model created by Nvidia, to accelerate the ArUCO marker detection process. By utilizing the computational power of GPUs, we aim to significantly improve the performance of the ArUCO marker detection algorithm, making it suitable for real-time applications on platforms like the Nvidia Jetson.

2. Motivation

In the field of robotics, ArUCO markers are frequently used for tasks such as object detection and pose estimation. The OpenCV library provides a convenient function, cv::aruco::detectMarkers, to facilitate this process. However, we have encountered performance issues on some systems, particularly those with limited CPU resources. Given that the Nvidia Jetson TX2, which we are using, is equipped with a powerful GPU, we aim to leverage CUDA for potential performance improvements.

During our investigation, we identified that a significant portion of the processing time is spent in repeated calls to cv::adaptiveThreshold. This gives us an opportunity for speed-ups by moving this computation to the GPU, thereby enhancing the overall efficiency of the marker detection process. By optimizing this function using CUDA, we hope to achieve faster and more reliable marker detection, which is critical for real-time robotic applications.

3. Problem Analysis

a. Introduction of Cuda

CUDA (Compute Unified Device Architecture) is a parallel computing platform and API created by NVIDIA. CUDA enables parallel execution of tasks across multiple threads, making it ideal for computationally intensive applications that can be parallelized. It leverages the thousands of processing cores within modern GPUs to perform tasks in parallel, leading to significant performance gains compared to traditional CPU-based processing.

b. Description of Image Thresholding

Image thresholding is a fundamental operation in image processing that aims to separate objects or regions of interest from the background based on pixel intensity values. The thresholding process involves comparing the intensity of each pixel in the image to a predefined threshold value and classifying the pixel as either foreground or background.
For example, we input an image as Figure 1, and get the result image as Figure 2. In this way, we mark edges for all objects and make it easier for the following image processing.

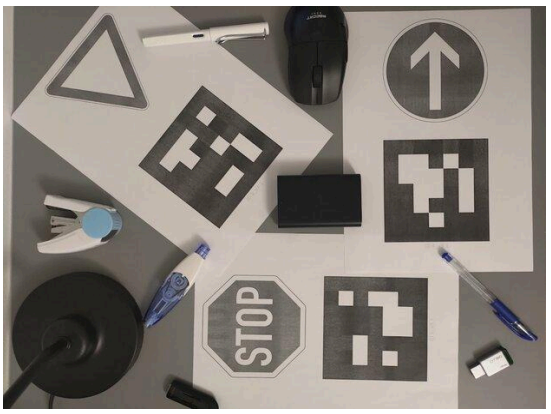

Figure 1

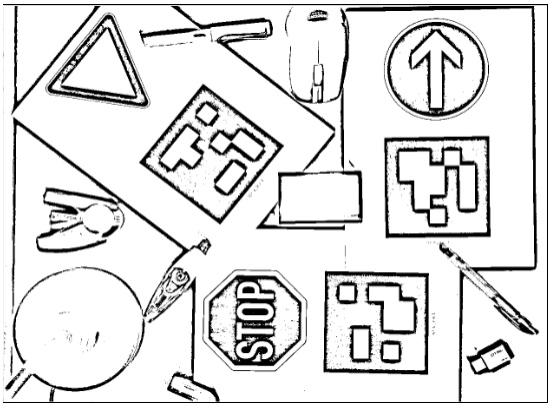

Figure 2

c. Computing for Threshold

Since the advantage of CUDA is that we can use parallel computing, we need to develop an algorithm that does parallel computing to do thresholding.
We can separate the Algorithm into two parts, CalculateMean and ApplyThreshold.

i. CalculateMean

The calculation of the mean is a crucial step in image thresholding, as it helps determine an appropriate threshold value for distinguishing foreground and background pixels. The mean value of pixel intensities within a local neighborhood or window is computed to capture the overall intensity characteristics of the image.

- **Algorithm**

  Launch CUDA kernel for mean value calculation, Each thread computes the mean value for a pixel in the image:
  For each pixel (i, j):
  - Iterate over a square window centered around the pixel with the specified block size.
  - For each pixel in the window:
    - Compute the sum of pixel values within the window.
  - Compute the mean value for the pixel by dividing the sum by the square of the block size.
  - Store the mean value in the destination array.

- **Demonstration**

  We take the left graph as input. For each pixel, we take the average value for a square window around the pixel with block size 3. The result is shown in the right graph (Fig. 3)

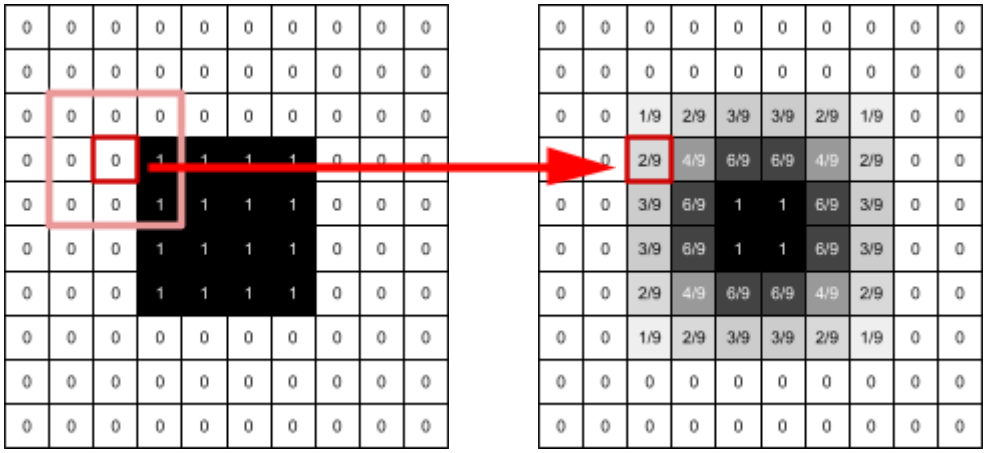

Figure 3

- **Time Complexity**

  Let n be the number of rows, m be the number of columns, and k be the block size, the time complexity of the algorithm is: $O(n \times m \times k^2)$

ii. Optimizing calculating the mean values

One of the optimization strategies employed in our CUDA implementation involves separating the calculation of mean values for rows and columns. Instead of computing the mean values for the entire image in a single operation, we divide the process into two separate steps: one for rows and one for columns.

- **Algorithm**

  First, we focus on the mean values for all rows. We compute the mean values of pixel intensities along rows for each pixel in the image. It iterates over each pixel (i, j) in the image and calculates the mean value using specified block sizes.

```
Launch the CUDA kernel for mean value calculation in the row, Each
thread computes the mean value for a pixel in the image:
    For each pixel (i, j) in the image:
        - Iterate over a window centered around the pixel
          with the specified block size (row):
            - Compute the sum value of the pixels in the row (i)
        - Compute the mean for the pixel values by dividing the sum
          by the block size
        - Store the mean value in the destination index
```

  Next, we work on the mean values for all columns. We compute the mean values of pixel intensities along columns using the precomputed row-wise mean values. Similar to the previous step, it iterates over each pixel (i, j) and calculates the mean value based on block sizes.

```
Launch the CUDA kernel for mean value calculation in the column, Each
thread computes the mean value for a pixel in the image:
    Using the result from the previous step,
    for each pixel (i, j) in the image:
        - Iterate over a window centered around the pixel
          with the specified block size (column):
            - Compute the sum value of the pixels in the column(j)
        - Compute the mean for the pixel values by dividing the sum
          by the block size
        - Store the mean value in the destination index
```

- **Demonstration**
  We take the image the same as the last part as an example. We first focus on the mean values for all rows (Fig. 4).

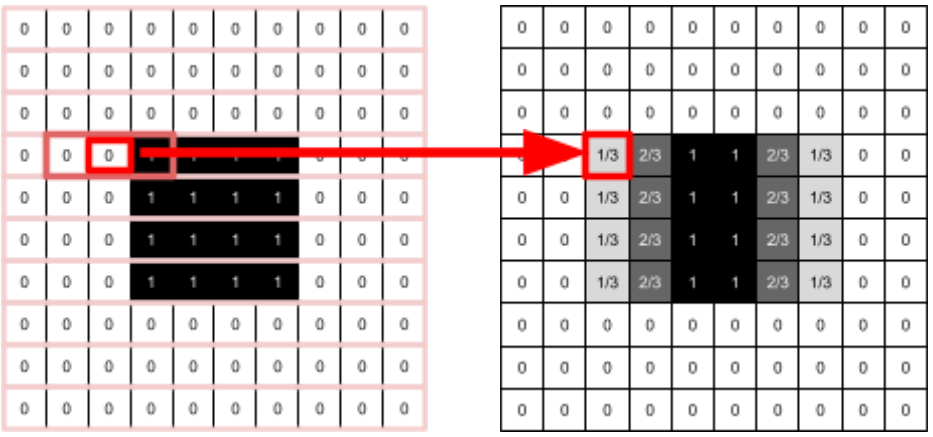

Figure 4

Then we take the result from the last step and focus on the mean values for all columns this time (Fig. 5).

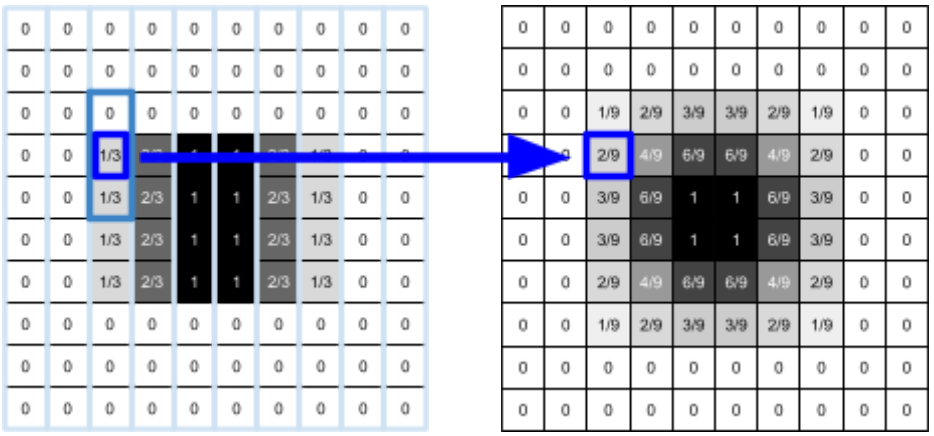

Figure 5

- **Time Complexity**
  Let n be the number of rows, m be the number of columns, and k be the block size, the time complexity of the algorithm is: $O(n \times m \times k)$.
  We can observe that while the result remains the same, the time complexity is improved.

iii. ApplyThreshold

After calculating mean values, the next step is to apply thresholds to classify pixels as foreground or background. Based on mean values and a constant, pixels above the threshold are classified as white (foreground), and others are classified as black (background).

- **Algorithm**

```
Launch CUDA kernel for threshold computing:
    For each pixel (i, j) in the image:
        Compute the threshold value using the mean value at (i, j) and
        the threshold constant c
        Update the corresponding pixel in the output image _dst based on
        the threshold condition
```

- **Demonstration**

We compare the original graph with the mean values from the previous step (Fig. 6).

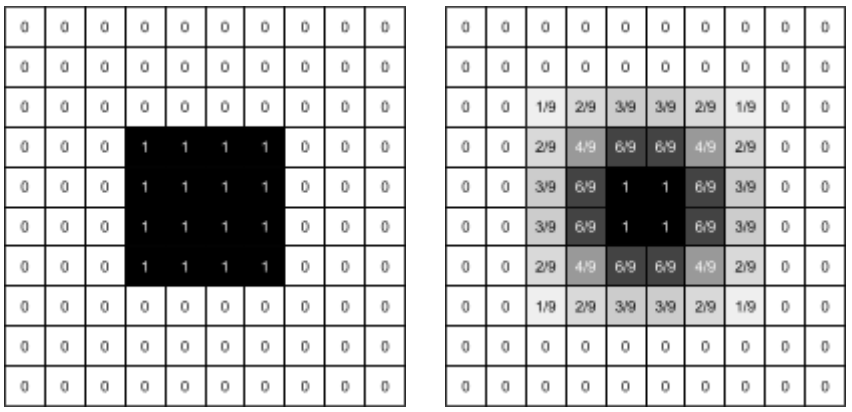

Figure 6

If the original value is greater or equal to the threshold, the pixel in the result is 1, else it's 0. This marks the object's edge (Fig. 7).

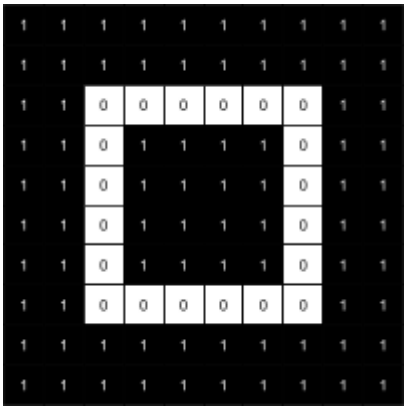

Figure 7

- **Time Complexity**

Let n be the number of rows, and m be the number of columns, the overall time complexity is: $O(n \times m)$

4. Implementation

a. Development Configuration

We use the NVIDIA Jetson TX2 Developer kit as our hardware.
The OS, OpenCV, and CUDA versions can be found in the table below.

Component	Version
OS	Ubuntu 18.04.6 LTS
OpenCV	3.4.20
CUDA	11.8

b. Program organization plan

The program is separated into 3 modules, main, aruco, and cu_aruco (Fig. 8).
The main module serves as the entry point for the program. For the aruco module, we modify the aruco module from OpenCV, to call functions in our own cu_aruco module for thresholding.
It is noticeable that we move out the code for allocating the VRAM for CUDA computing since we notice this step can also take a long time. Since it doesn't make sense to reallocate the VRAM and free up the memory for every literation, we do it only once before the for-loop. and only free it up at the end of the program.

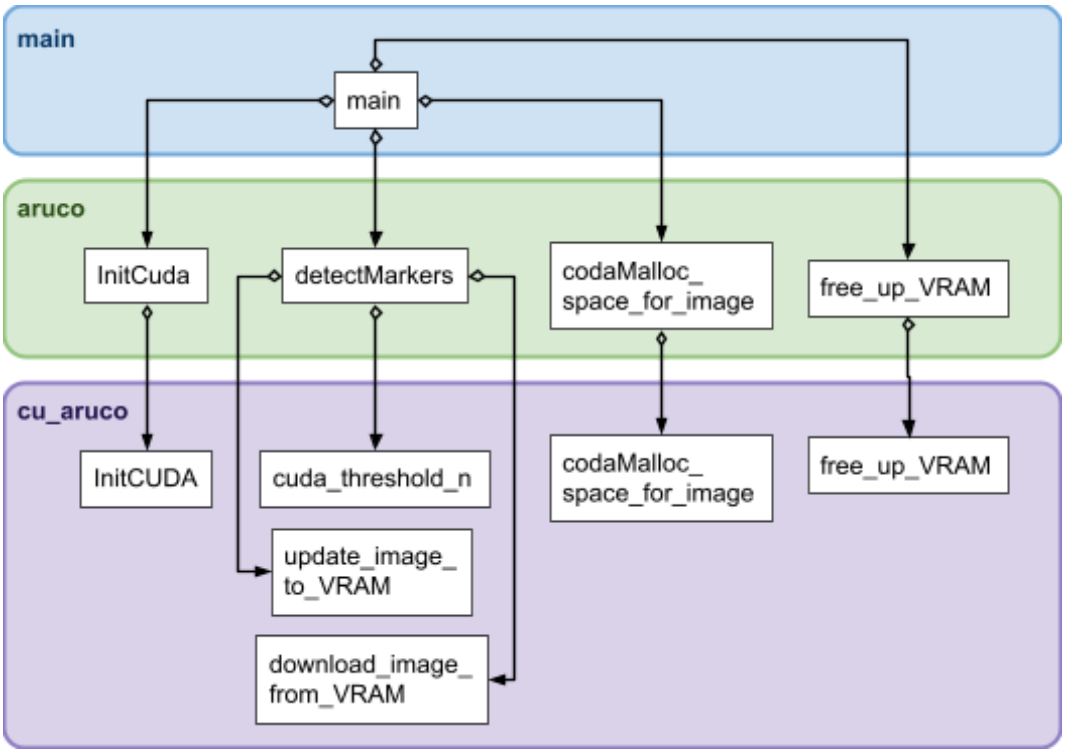

Figure 8

c. Code Implementation

i. main

The main function initializes marker detection and GPU acceleration using the ArUco library and CUDA, respectively. It continuously captures frames from the camera, detects markers in each frame, and displays the processed images in a window. The program terminates upon user input, releasing resources and closing all windows.

```
int main() {
    // Create an instance of the Aruco class
    aruco::Aruco aruco;

    // init Cuda
    aruco.initCuda();

    //init paramaters
    cv::VideoCapture cap = create_capture(1280, 720, 10);
    ...

    //malloc VRAM
    ...

    while(true) {
        // Capture a frame from the camera
        cap >> frame;

        // Check if the frame was captured successfully
        ...

        //Call the aruco.detectMarkers function
        aruco.detectMarkers(frame, dictionary, markerCorners,
                                    markerIds,parameters, cv::noArray());

        // Display the frame in the window
        cv::imshow("Camera Feed", frame);

        //Check for user input to exit
        ...
    }
    cap.release();

    //free up VRAM and destroy all Windows
    aruco.free_up_VRAM();
    cv::destroyAllWindows();

    return 0;
}
```

ii. aruco

- **_detectInitialCandidates**

  Here we use a `CUDA_IMPLE` flag to switch between the CUDA code and the original Opencv code.
  For the original Opencv code, it uses a `parallel_for_` to generate the threshold images and find the marker contours.
  Since we want to focus on improving the threshold part, we define our own `_cudaThreshold_n` function. The findMarkerContours function remains unchanged within a `parallel_for_` loop.

```
...
#if CUDA_IMPLE == 0
    // this is the parallel call for the previous commented loop
    parallel_for_(Range(0, nScales), DetectInitialCandidatesParallel(
                    &grey, &candidatesArrays,&contoursArrays, params));
#elif CUDA_IMPLE == 1
    //init paramaters
    ...

    //Copy the image to the VRAM
    cudaProcessor.update_image_to_VRAM(grey.data, d_src, dataSize);

    //call the _cudaThreshold_n function
    _cudaThreshold_n(grey, &thresh[0], params);

    //call the _findMarkerContours function using cv::parallel_for_
      in range 0 to nScales
    ...
#endif //CUDA_IMPLE
...
```

- **_cudaThreshold_n**

  For the `_cudaThreshold_n` function, we initial all the variables and call the `cudaProcessor.cuda_threshold_n`. After finishing computing the threshold images, we download the results from VRAM

```
static void _cudaThreshold_n(InputArray _in, Mat _out[],
                                const Ptr<DetectorParameters> &params){
    //init parameters
    ...

    cudaProcessor.cuda_threshold_n(d_src, d_dst, rows, cols,
                    step, winSize, nScales, params->adaptiveThreshConstant);

    //download the results from VRAM
    for(int i = 0;i < nScales;i++){
        cudaProcessor.download_image_from_VRAM(_out[i].data,
                                                d_dst + i * dataSize, dataSize);
    }
    //free up space
    ...
}
```

iii. cu_aruco

- **cuda_threshold_n**

  The `cuda_threshold_n` function utilizes CUDA to perform thresholding on an input image stored in GPU memory. It first initializes the necessary CUDA parameters, allocates memory spaces, and calculates mean values for each pixel neighborhood. Subsequently, it applies a thresholding operation based on the computed mean values. Finally, it synchronizes device execution and frees up allocated memory spaces.

```
void CudaProcessor::cuda_threshold_n(const unsigned char* d_src,
unsigned char* d_dst, int rows, int cols, int step, int* winSize, int nScales,
double constant) {

    //init blocks and threads
    dim3 blocks(iDivUp(rows, 16), iDivUp(cols, 16));
    dim3 threads(16, 16);

    //init paramaters
    ...

    //malloc VRAM spaces for cuda function
    ...

    //calculate mean values
    calculateMeanRows<<<blocks, threads>>>(d_src, d_meanRows,
                                           rows, cols, winSizeDevice, nScales);

   calculateMeanCols<<<blocks, threads>>>(d_meanRows, d_mean,
                                           rows, cols, winSizeDevice, nScales);


    //apply threshold
    applyThreshold_n<<<blocks, threads>>>(d_src, d_dst, d_mean,
                                                rows, cols, constant, nScales);


    // Wait for all kernels to finish
    cudaDeviceSynchronize();

    //free up memory spaces
    ...
}
```

- **calculateMeanRows & calculateMeanCols**
  These CUDA kernels compute the mean values for each pixel in an image, considering both rows and columns. They operate on the GPU, with each thread processing a single pixel. The kernels iterate over the image rows and columns, calculating the mean within a specified neighborhood determined by the provided block sizes. The calculateMeanRows kernel computes the row-wise means, while the calculateMeanCols kernel computes the column-wise means. The computed mean values are stored in the output array.

```
__global__ void calculateMeanRows(const unsigned char* _src,
        int* _mean, int rows, int cols, int blockSizes[], int numBlockSizes) {

   int i = blockIdx.x * blockDim.x + threadIdx.x;
   int j = blockIdx.y * blockDim.y + threadIdx.y;
   if (i < rows && j < cols) {
      int sum = 0;
      int count = 0;

      for (int bS = 0; bS < numBlockSizes; bS++) {
         for (int x = -blockSizes[bS] / 2; x <= blockSizes[bS] / 2; x++) {
            int col = j + x;
            if (col >= 0 && col < cols && (bS == 0
               || (x < -blockSizes[bS - 1] / 2)  || (x > blockSizes[bS - 1] / 2))) {
               sum += _src[i * cols + col];
               count++;
         }}
         _mean[(i * cols + j) + bS * rows * cols] = sum / count;
}}}
```

```
__global__ void calculateMeanCols(const int* _meanRows, int* _mean,
                        int rows,int cols, int blockSizes[], int numBlockSizes) {

   int i = blockIdx.x * blockDim.x + threadIdx.x;
   int j = blockIdx.y * blockDim.y + threadIdx.y;
   if (i < rows && j < cols) {
      int sum = 0;
      int count = 0;

      for (int bS = 0; bS < numBlockSizes; bS++) {
         for (int y = -blockSizes[bS] / 2; y <= blockSizes[bS] / 2; y++) {
            int row = i + y;
            if (row>=0 && row<rows && (bS==0
               || (y< -blockSizes[bS - 1] / 2) || (y > blockSizes[bS - 1] / 2))) {
               sum += _meanRows[row * cols + j + bS * rows * cols];
               count++;
         }}
         _mean[(i * cols + j) + bS * rows * cols] = sum / count;
}}}
```

- **applyThreshold_n**
  This CUDA kernel applies thresholding to an input image based on the computed mean values. Operating on the GPU, each thread processes a single pixel in the image. The kernel iterates over the image rows and columns, computing the threshold for each pixel at multiple scales. If the intensity of the pixel exceeds the threshold, the corresponding pixel in the output image is set to 0; otherwise, it is set to 255. The threshold value is determined by subtracting a constant value `c` from the mean value computed for each pixel.

```
__global__ void applyThreshold_n(const unsigned char* _src,
unsigned char* _dst, int* _mean, int rows, int cols,
double c, int nScales) {

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if (i < rows && j < cols) {
        for (int k = 0; k < nScales; ++k) {

            int threshold = _mean[(i * cols + j) + k * rows * cols] - c;

            _dst[k * rows * cols + i * cols + j] =
                                        (_src[i * cols + j] >= threshold) ? 0 : 255;

        }
    }
}
```

5. Interpretation of Results

a. Test Image and the Result of the Program

We created a test image containing ArUCO markers from three different dictionaries, as well as objects with no markers at all (Fig. 9).

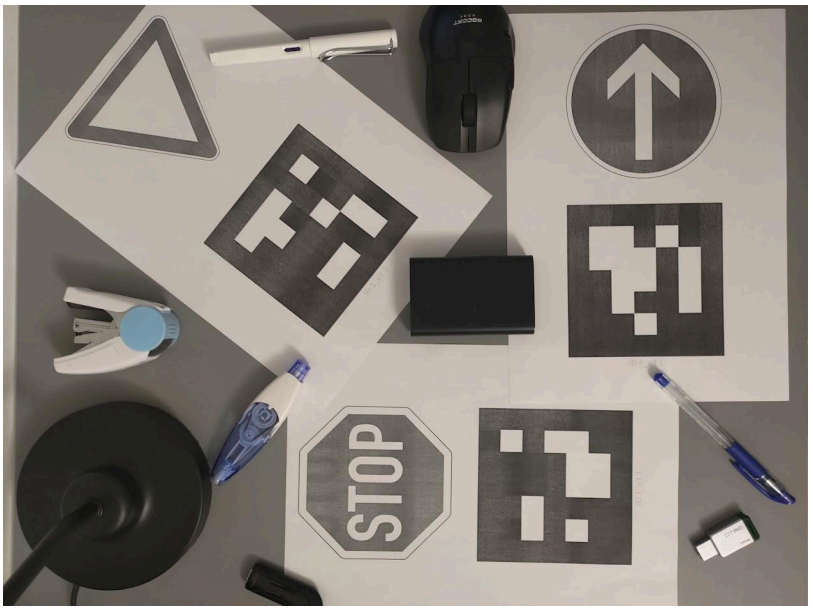

Figure 9

In the results, we can see that all markers were detected correctly (Fig. 10).

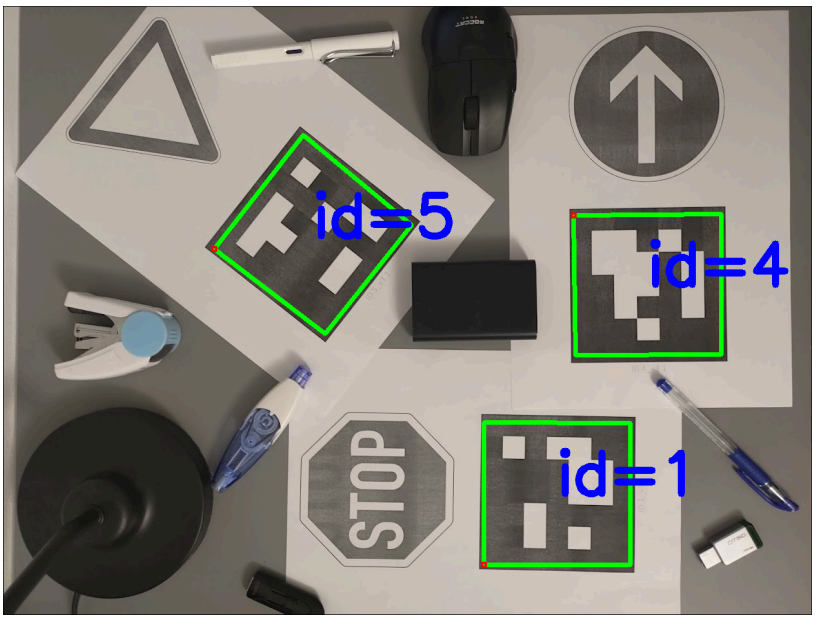

Figure 10

b. Time Analysis

We created a test program and generated test images at different resolutions. We added timestamps before and after the detectMarkers function to measure the processing time. Each image was processed 10 times, and we calculated the average time used. We then used Gnuplot to convert the data into graphs.
First, on the Jetson, we built OpenCV and the project in debug mode. The result is shown in the picture below (Fig. 11).

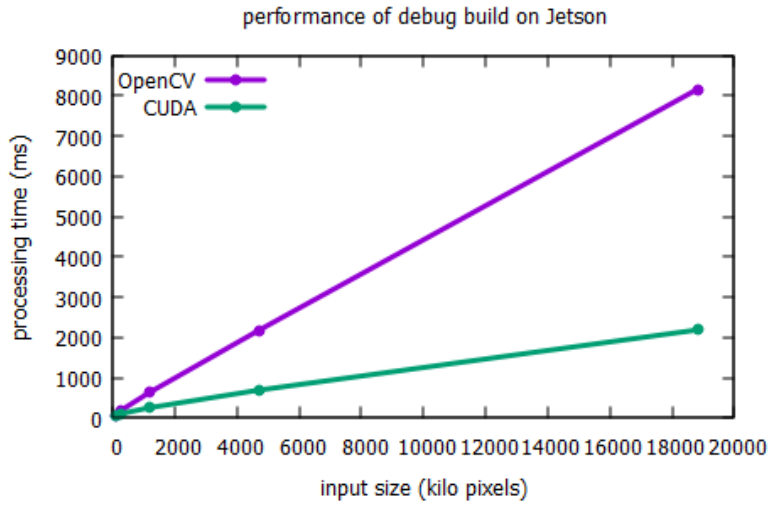

Figure 11

We then built OpenCV and the project again in release mode. The result is shown below (Fig. 12).

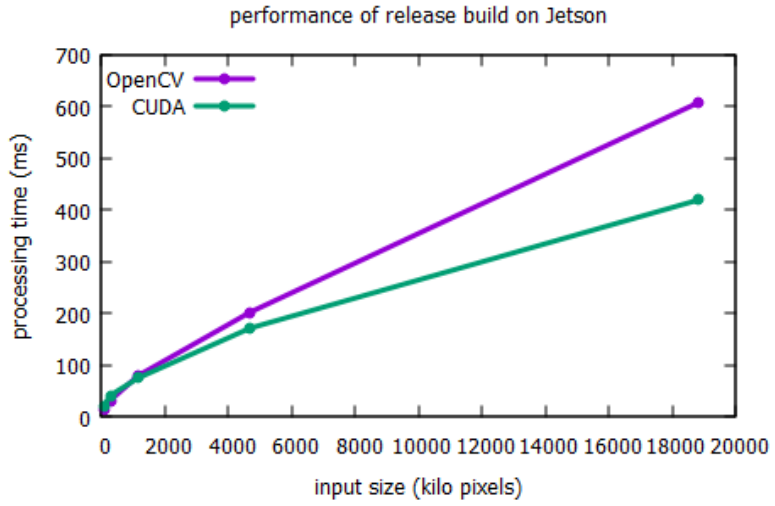

Figure 12

We observed that the difference between the OpenCV reference CPU implementation and our CUDA implementation is much more pronounced in debug mode. However, in release mode, while the trend remains visible, the performance gap is reduced. In both modes, the processing time for the CUDA implementation is faster, especially for large image sizes.

We tested the program again on a different device, a laptop with the hardware information shown in the table below.

Component	model
CPU	Intel Core i7-8750H
GPU	NVIDIA GeForce GTX 1070 with Max-Q Design

In debug mode, the result is shown below (Fig. 13)

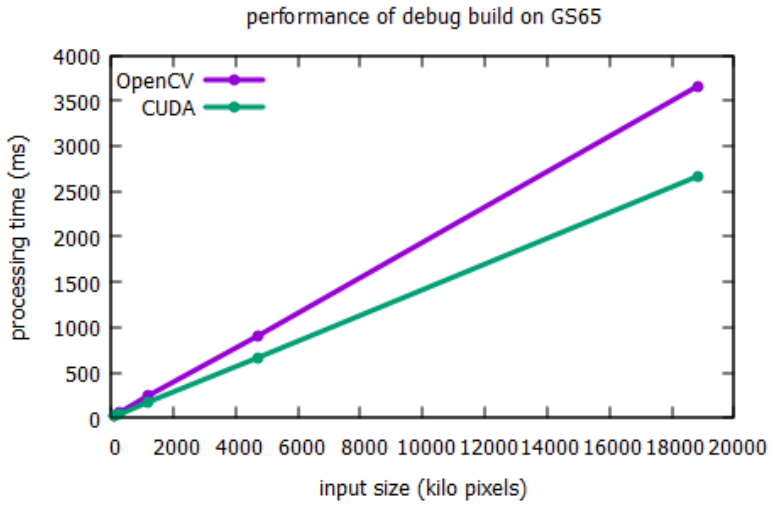

Figure 13

We rebuilt the project in release mode, and the result is shown below (Fig. 14).

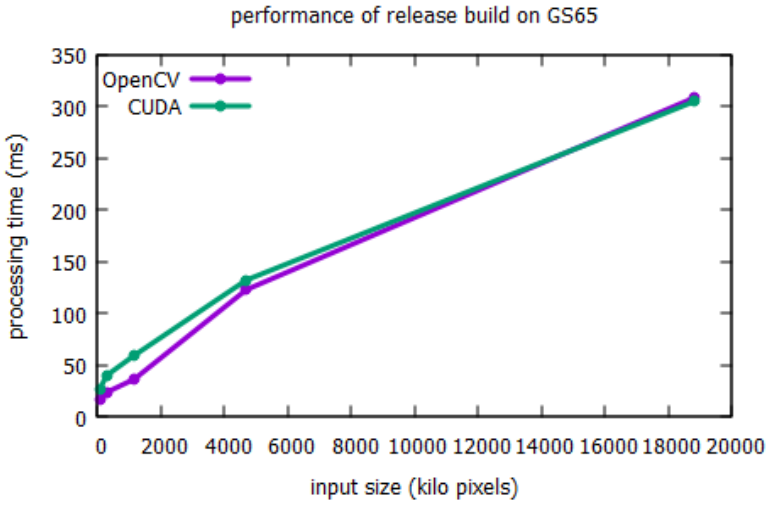

Figure 14

On the laptop, the CPU is more powerful than the Jetson. With strong compiler optimizations, in release mode, the performance gap between OpenCV and CUDA implementations is minimal. However, due to parallel computing advantages, the performance of CUDA implementation is slightly better for larger image sizes.

6. Conclusion

In this project, we aimed to enhance the performance of the cv::aruco::detectMarkers function in OpenCV by leveraging the GPU capabilities of the Nvidia Jetson TX2. Through analysis, we identified that the cv::adaptiveThreshold function was slowing down the process due to its repeated execution. By moving this computation to CUDA, we were able to improve the processing speed of ArUCO marker detection.

Our experimentation involved testing the thresholding function on images of different resolutions and analyzing the execution time of both the original CPU-based approach and the optimized CUDA implementation. We observed that for images with higher resolutions, the CUDA implementation demonstrated superior performance, showcasing the effectiveness of parallel computing in accelerating image processing tasks.

However, We observed that the difference between the OpenCV reference CPU implementation and our CUDA implementation is much more pronounced in debug mode. Also, on a laptop with a more powerful CPU than the Jetson, the performance gap between the OpenCV and CUDA implementations is minimal. Further optimizations and fine-tuning may still be required to maximize the efficiency of the CUDA implementation across different use cases and scenarios.

In conclusion, this project provides valuable insights into the optimization of image thresholding using CUDA parallel computing. By using the computational power of GPUs, we can enhance the performance of image processing algorithms, opening up possibilities for real-time applications and advanced computer vision systems.